第5回　Python I：Jupyter、基本文法と現在価値

script・対話モード・notebook、変数・if・for・関数、複利と現在価値

2026年10月15日（木）2限　金融論

実習の準備

年率5%は `rate=0.05`、期間5年は `years=5` と入力します。金額は円、受取は各年末です。年率と年数をそろえて計算します。

error実習の後は、意図的に壊した行を戻すかコメントにし、kernelを再起動して先頭から実行し、fileを保存します。

この回で扱うこと

- プログラミング言語とは：コンパイル型とスクリプト型、代表的な言語
- Pythonを動かす3つの方法：script（`.py`）、対話モード、notebook（`.ipynb`）
- 変数、型、計算
- 条件分岐（`if`）
- listと繰り返し（`for`）
- 関数（`def`）
- 複利と現在価値

手を動かすのは `workbook/workbook.py` です。この回の3〜7節のcodeを上から並べたscriptで、▷ を押すと全部が実行されます。同じcodeを `# %%` でcellに区切り、Shift+Enterで1つずつ実行しながら説明を読める形が `workbook/workbook_cells.py`、notebook版が `workbook/workbook.ipynb` です。`workbook/pv.py` は、scriptとして動かすための短いfileです。`workbook/scripts/` には、3〜7節を節ごとに独立した `.py` にしたものがあります。「やってみる」の答え合わせは、Codexに頼めます。

プログラミング言語とは

プログラミング言語は、コンピュータにさせたい処理を人が書くための言葉です。書いたものはただのtext fileで、これをprogram、code、またはsourceと呼びます。

```
amount = 110
rate = 0.10
pv = amount / (1 + rate)
print(pv)
```

上の4行はPythonで書いたprogramです。コンピュータが直接理解できるのは0と1の並び（機械語）だけなので、人が書いたcodeを機械語に直す仕組みが要ります。その直し方で、言語は大きく2つに分かれます。

コンパイル型とスクリプト型

コンパイル型は、codeを先にまるごと機械語へ翻訳し、できた実行file（Windowsなら `.exe`）を動かします。翻訳する道具をcompilerと呼びます。翻訳に手間がかかるかわりに、できたprogramは速く動きます。C、C++、Go、Rustがこの型です。OSやゲーム、証券取引所の売買systemはこの型で書かれています。

スクリプト型は、codeを翻訳せず、interpreterと呼ばれるprogramが1行ずつ読んで実行します。書いてすぐ動かせるので、試しながら書くのに向いています。そのかわり、同じ処理ならコンパイル型より遅くなります。Python、R、JavaScript、Excelのマクロ（VBA）がこの型です。Pythonはスクリプト型なので、次の節の「1行打つたびに実行する」使い方ができます。

中間の型もあります。Javaは、codeをbytecodeという機械語に近い中間の形へ先に翻訳し、JVM（Java Virtual Machine）というprogramがそれを実行時に機械語へ直しながら動かします。どのOSでも同じbytecodeが動くのが利点です。C#も同じ仕組みです。Pythonも内部ではbytecodeへ直してから実行しますが、その翻訳は自動で毎回行われるので、使う側からはスクリプト型として見えます。

Juliaは、実行時に必要な関数を機械語へ翻訳するJIT（Just-In-Time）コンパイルを使います。翻訳の途中でLLVM IRという中間表現を経由します。対話的にcodeを動かせて、翻訳後の計算は高速に実行できます。

Pythonの速さが足りないところは、内部をCで書いたlibraryが補います。第6回以降で使うpandasやnumpyがそれで、表計算の部分はCの速さで動きます。

代表的な言語

言語	型	主な用途
C / C++	コンパイル	OS、組み込み、高速な計算
Java	中間（bytecode + JVM）	銀行や企業の業務system、Androidアプリ
JavaScript	スクリプト	Webページの動き
Python	スクリプト	データ分析、機械学習、AI,（web開発など汎用言語）
R	スクリプト	統計、経済学・金融の実証研究
Stata	スクリプト	経済学の実証研究。労働・開発・医療経済学の論文で標準、商用
MATLAB	スクリプト	マクロ経済学の動学モデル、商用
Julia	JITコンパイル（実行時に機械語へ翻訳）	マクロの動学モデル、計算量の多い構造推定
Fortran	コンパイル	異質的個人など大型マクロモデル
SQL	（問い合わせ言語）	databaseからデータを取り出す
VBA	スクリプト	Excelの自動化

Pythonは1991年に公開された言語で、文法が短く読みやすいことから、いまはデータ分析とAIの標準的な言語になっています。Rは統計のために作られた言語で、経済学の論文の計算はRで書かれたものが多くあります。

経済学では、分野ごとに使う言語が分かれています。個票データで回帰分析をする実証研究はStataかRです。Stataは有料で、回帰分析のcommandが1行で済むので、労働経済学や開発経済学の論文の再現fileはStataで配られることが多いです。マクロ経済学の動学モデルを解く研究はMATLAB（東北大ライセンスあり）がよく使わ

れます。近年はこの用途をJuliaやPythonへ移す動きがあります。金融のデータ分析は、業界ではPythonが標準です。この授業では第5回から第12回でPython、第13回以降でRを使います。

言語が違っても、変数・条件分岐・繰り返し・関数という部品は共通です。この回でPythonの部品を覚えると、別の言語に移るときは書き方の違いを見るだけで済みます。

library・import・pip

Python本体に入っているのは、この回で使う変数・`if`・`for`・関数と、`print` や `round` のような少数の関数だけです。CSVを表として読む、図を描く、といった機能は、他の人が書いてまとめたcodeの束を借りて使います。この束をlibrary、またはpackageと呼びます。

libraryは、pipという道具で環境に入れます。`pip install pandas` のように打つか、Codexに「pandasを入れて」と頼めば済みます。入れるのは環境ごとに1回です。HW01で `finance-env` にpandasとmatplotlibを入れたのがこの作業でした。この授業で使うのは、表を扱うpandas、図を描くmatplotlib、数値計算のnumpyの3つです。

`import` は、入れてあるlibraryを、いま動いているPythonに読み込む操作です。scriptならfileの先頭に書き、notebookなら最初のcellに書いて、kernelを再起動するたびに実行し直します。次の節で最初に実行する `import sys` は、Python本体に付いてくるlibrary `sys` を読み込む行です。`import pandas as pd` のように `as` を付けると、以後 `pd` という短い名前で呼べます。`pd` は慣例の略で、pandasを使う人はほぼ全員この名前を使っています。

入れていないlibraryを `import` すると `ModuleNotFoundError` になります。別の環境を選んでいるときにも同じerrorが出ます。次の節で環境を選ぶのは、このためです。

Pythonを動かす3つの方法

VS Codeで使うPython環境を選ぶ

Pythonのenvironment（環境）は、Python本体と、そこに入れたlibraryの組み合わせです。同じPCに複数の環境がある場合、別の環境を選ぶと、入れたはずのlibraryが見つからないことがあります。この授業では、HW01で作った `finance-env` を使います。すでに選ばれていれば、選び直す必要はありません。

Pythonファイル（`.py`）を開くと、画面右下のステータスバーにPythonのバージョンや環境名が表示されます。その表示をクリックし、出てきた候補から `finance-env` を選びます。

表示が見つからない場合は、Windowsでは `Ctrl + Shift + P`、Macでは `Command + Shift + P` でコマンドパレットを開きます。画面上部に出る検索欄に `Python: Select Interpreter` と入力して選ぶと、Python環境の候補が出ます。Python拡張機能が必要なので、この項目が出ない場合はCodexに導入を頼めます。

`finance-env` が候補にない場合は、`Enter interpreter path...` から、その環境のPythonを指定します。HW01の配置なら、`金融論` フォルダ内の、Windowsでは `finance-env\Scripts\python.exe`、Macでは `finance-env/bin/python` です。実際の保存先が分からない場合は、Codexに既存の `finance-env` を探してもらえます。

環境を切り替えた後は、新しいterminalを開くと、選んだ環境が通常は有効になります。以下のcodeを実行すると、実際に使っているPythonの場所が表示されます。pathに `finance-env` が含まれていれば、授業用の環境で動いています。

```
import sys
print(sys.executable)
```

画面上部に一時的に出る文字入力用の欄は、入力ボックス（Input Box）です。コマンドパレットは操作を検索する欄で、実習中の入力ボックスは値を入力する欄です。表示されている案内を読むと、何を入力する場所かが分かります。

script

scriptは、codeだけを書いたtext fileです。拡張子は `.py` です。

```
# 1年後に受け取る110円の現在価値（割引率10%）

amount = 110
rate = 0.10
years = 1

pv = amount / (1 + rate) ** years
print(round(pv, 2))
```

1行目の `#` から行末まではコメントで、Pythonは読み飛ばします。人が読むための説明をここに書きます。行の途中に `# ...` と付けて、その行の説明にすることもできます。実行させたくない行の先頭に `#` を付けると、その行は消さずに止めておけます。VS Codeでは、行を選んで ⌘/（WindowsはCtrl+/）を押すと、選んだ行の先頭に `#` が付きます。もう一度押すと外れます。ワークブックでerrorを確かめた行は、あとでこの形にしておきます。

terminalで `python pv.py` と打つか、VS Codeの右上の ▷ を押すと、上から下まで一度に実行されます。結果はterminalに表示され、fileには残りません。

VS Codeでは、実行したい行をマウスで選んで Shift+Enter を押すと、その部分だけをterminalの対話モードで実行できます。右クリックして「ターミナルで選択範囲/行を実行」を選んでも同じです。何も選択せずに Shift+Enter を押すと、カーソルがある1行が実行されます。`def` や `for` のように字下げで続く複数行の中にカーソルがあれば、そのまとまり全体が送られます。

Python 3.13の対話モードは、複数行をまとめて貼り付けると文字が崩れて `SyntaxError` になることがあります。terminalの環境変数 `PYTHON_BASIC_REPL` を `1` にしておくと、以前の対話モードになり、まとめて送れます（環境変数は第7回で扱います）。Codexに「VS Codeのterminalで環境変数 PYTHON_BASIC_REPL=1 が常に有効になるように設定して」と頼むと、VS Codeの設定fileに書いてくれます。

実行のたびに、何もない状態から始まります。前回の実行で作った変数は残っていません。

対話モード

terminalで `python` とだけ打つと、`>>>` が表示されます。Pythonが起動したまま、1行打つたびに実行して結果を返す状態です。

```
>>> amount = 110
>>> amount * 2
220
>>> exit()
```

式を打つと、`print` しなくても値が表示されます。作った変数は `exit()` で抜けるまで残り、抜けると消えます。打ったcodeも結果も、どこにも保存されません。

`>>>` の後ろに打つのはPythonのcodeです。ここで `python pv.py` と打つと `SyntaxError` になります。`python pv.py` はterminalに打つcommandなので、`exit()` で抜けてから打ちます。

notebook

notebookは、文章・code・実行結果を1つのfileにまとめたものです。拡張子は `.ipynb` です。

中身はcellに分かれています。文章を書くMarkdown cellと、codeを書くcode cellの2種類です。code cellは1つずつ実行でき（Shift+Enter）、結果はそのcellのすぐ下に出て、fileに保存されます。

notebookの後ろでは、kernelと呼ばれるPythonが動き続けています。あるcellで作った変数は、kernelに残り、別のcellから使えます。値が自動で表示されることも含めて、kernelの動きは対話モードと同じです。notebookは、対話モードで打つcodeをcellに分け、codeと結果をfileに残せるようにしたものです。

notebookでは、右上の `Select Kernel`（選択済みならkernel名）をクリックして、使うPythonを選びます。`Select Another Kernel...` が出た場合はそこから `Python Environments` を開き、`finance-env` を選びます。候補や選択中のkernel名で、環境を確認できます。右下のPython環境の選択とは別なので、notebookの右上でも確認します。

その環境に `ipykernel` というpackageが入っていれば、notebookを実行できます。VS CodeがInstallを勧めた場合は、選んだ `finance-env` にインストールします。準備や候補への表示で詰まった場合は、Codexに既存の `finance-env` を使うよう伝えて設定を頼めます。

最初のcode cellで `import sys` と `print(sys.executable)` を実行すると、kernelが使っているPythonの場所も確かめられます。

違い

	script（`.py`）	対話モード	notebook（`.ipynb`）
始め方	`python pv.py`、▷、Shift+Enter	`python`	`.ipynb` を開いてcellを実行
実行する単位	file全体、または選択行	1行ずつ	cell 1つ
実行の順番	いつも上から下	打った順	cellを押した順
変数	実行のたびに空から始まる	`exit()` まで残る	kernelに残り続ける
値の表示	`print` したものだけ	自動	cellの最後の行は自動
codeと結果	codeはfileに残る。結果はterminalに出るだけ	どちらも残らない	どちらもfileに残る
向いている使い方	同じ処理を何度も走らせる、他の人が再実行する	1行だけ試す	試しながら読む、説明と結果を並べる

notebookの実行順序

新しいnotebookを作り、cellを動かす順番で結果がどう変わるかを試します。

VS Codeの左のファイル一覧で `workbook` フォルダを選び、「新しいファイル」から `lecture05_live.ipynb` を作ると、空のnotebookが開きます。右上の `Select Kernel` から `finance-env` を選びます。

最初のcode cellに次のcodeを入力し、Shift+Enterで実行します。

```
# cell 1
price = 100
```

次のcode cellに以下を入力し、Shift+Enterで実行すると `200` が出ます。cellは上部の `+ Code` でも追加できます。

```
# cell 2
price * 2
```

次にcell 1を `price = 300` に書き換え、cell 1は実行せずにcell 2だけ実行すると、やはり `200` が出ます。kernelの中の `price` は100のままだからです。cell 1を実行してからcell 2を実行すると、結果は `600` に変わります。

notebookの上部にある `Restart` を押してから `Run All` を押すと、変数が空の状態から画面のcodeを上から順に実行します。この例では `600` が出ます。表示されているcodeと結果をそろえられます。

⌘S（WindowsはCtrl+S）で保存すると、入力したcodeと実行結果が `lecture05_live.ipynb` に残ります。

この授業での使い分け

notebookがあまり得意ではないので、第5回と第6回は、`.py` を主に使います。▷ で上から全部を実行するのがscript、`# %%` で区切った版をShift+Enterで1cellずつ実行するのがnotebookと同じ使い方で、どちらも同じcodeです。第10回では、何度も使う処理を関数にまとめ、`.py` に置いていきます。HW02で提出するcodeは、他の人が上から実行して同じ結果になる形にします。

変数、型、計算

代入

`=` は「右の値を左の名前に入れる」という意味です。

```
A = 1
B = 2
C = 3
print(A, B, C)
```

等しいかどうかを調べるときは `==` を使います。`test == 60` は、`test` が60なら `True`、違えば `False` を返します。

計算

書き方	意味	例	結果
`+` `-` `*`	足し算、引き算、掛け算	`10 * 20 + 100`	`300`
`/`	割り算	`7 / 2`	`3.5`
`` `| べき乗 | 1.1 2 | 1.2100000000000002 | | round(x, 2) | 小数第2位で丸める | round(1.1 ** 2, 2) | 1.21` ``			

べき乗は `**` です。`^` は別の意味の記号で、`1.1 ^ 2` はerrorになります。

小数は2進数で近似して計算するので、`1.1 ** 2` の末尾に `...0002` が付きます。表示するときは `round` で丸めます。

型

値には型があります。`type()` で調べられます。

型	中身	例
`int`	整数	`3`
`float`	小数	`3.0`、`0.05`
`str`	文字列	`"Pen Pineapple Apple Pen"`、`"3"`
`bool`	真偽	`True`、`False`

`"3"` は数字に見えても文字列です。`"3" + "3"` は `"33"` になり、文字をつなげます。`"3" + 3` はerrorです。数として足すには `int("3")` や `float("3")` で数に変換します。

CSVから読んだ値が文字列になっていて計算が合わない、ということが第6回以降でよく起きます。

条件分岐

テストの点数が60点以上なら “Pass”、それ以外なら “Fail” と表示します。

```
test = 61
if test >= 60:
    print("Pass")
else:
    print("Fail")
```

`if` の行の最後に `:` を付け、次の行を字下げ（半角空白4つ）します。字下げされた行が、条件が成り立ったときに実行される部分です。

3つ以上に分けるときは `elif` を使います。

```
test = 60
if test > 60:
    print("Pass")
elif test == 60:
    print("Just barely Pass")
else:
    print("Fail")
```

比べる記号は `>`、`<`、`>=`、`<=`、`==`、`!=`（等しくない）です。

listと繰り返し

list

複数の値を `[ ]` でまとめたものがlistです。

```
values = [A, B, C]
values[0]
```

Pythonの番号は0から始まります。 `values[0]` が1番目、`values[2]` が3番目です。`values[3]` はerrorになります。

listに数を掛けると、要素ごとの計算ではなく、listの繰り返しになります。`[2, 3, 4] * 2` は `[2, 3, 4, 2, 3, 4]` です。列ごとの計算は、第6回のpandasで扱います。

for

同じ処理を、値を変えながら繰り返します。1から5までを順に足し、途中の結果を表示します。

```
total = 0
for i in range(1, 6):
    total = total + i
    print(total)
```

`range(1, 6)` は1, 2, 3, 4, 5です。終わりの6は含みません。 `range(1, 14, 3)` は3ずつ増えて1, 4, 7, 10, 13になります。

`for` も `if` と同じく、`:` と字下げで繰り返す範囲を示します。

複利

100円を年10%で2年間預けます。

- 単利は、毎年元金100円に対して10円の利子が付きます。2年後は120円です。
- 複利は、利子にも利子が付きます。1年目に110円、2年目は110円の10%で11円が付いて121円です。

T 年後の金額は、単利なら $P(1+rT)$、複利なら $P(1+r)^T$ です。20年にすると、単利は300円、複利は約672.75円になります。

複利は「前の年の金額に $(1+r)$ を掛ける」の繰り返しなので、`for` で書けます。途中で金利が変わる場合は `if` を組み合わせます。

```
fuku = 100
for year in range(1, 21):
    if year <= 10:
        fuku = fuku * 1.1
    else:
        fuku = fuku * 1.2
print(round(fuku, 2))
```

最初の10年が10%、残りの10年が20%のとき、20年後は約1,605.98円です。

関数

何度も使う計算は、関数にして名前を付けます。

```
def future_value(principal, rate, years):
    return principal * (1 + rate) ** years

future_value(100, 0.10, 2)
```

`def` の後ろが関数の名前、`( )` の中が引数（関数に渡す値）です。`return` の後ろが、関数から返ってくる値です。

`print` と `return` は違います。`print` は画面に表示するだけで、値を返しません。`return` で返した値は、別の変数に入れたり、さらに計算に使ったりできます。

```
x = future_value(100, 0.10, 2)
x * 2
```

現在価値

1回だけ受け取る

複利の式を逆向きに使います。T 年後に受け取る C 円を、割引率 r で今の価値に直すと

$$PV = \frac{C}{(1+r)^T}$$

です。1年後の110円を10%で割り引くと、$110/1.1 = 100$ 円です。今の100円を10%で1年運用すると110円になるので、1年後の110円と今の100円は同じ価値だと考えます。

```
def present_value(amount, rate, years):
    return amount / (1 + rate) ** years

round(present_value(110, 0.10, 1), 2)
```

割引率が高いほど、また受け取りが先になるほど、現在価値は小さくなります。割引率が0なら、将来の金額と現在価値は同じです。

何年も受け取る

毎年1万円を5年間受け取る権利の現在価値は、各年の現在価値の合計です。割引率は5%とします。

このとき、いつ受け取るかを決めておく必要があります。

受け取る時点	現在価値の合計
1年後から5年後	約43,294.77円
今すぐ（0年後）から4年後	約45,459.51円

最初の1回を今受け取るほうが、割り引かれない分だけ大きくなります。

```
total = 0
for t in range(1, 6):
    total = total + present_value(10000, 0.05, t)
print(round(total, 2))
```

株価を、将来の配当と売却価格の現在価値として考える話は、第7回で扱います。

この回の要点

- scriptは上から全部を毎回実行します。対話モードとnotebookは、実行した変数が残ります。notebookの結果は、Restart と Run All で確かめられます。
- `if` と `for` は `:` と字下げで範囲を示します。Pythonの番号は0から始まり、`range` は終わりを含みません。
- 現在価値は $C/(1+r)^T$ です。何年も受け取るときは、受け取る時点を決めてから合計します。